

Concordia University

Lab 2: The Discrete Time Fourier Transform and Introduction to Simulink

ELEC 342
Lab Section: UL-X

Discrete Time Signals and Systems

Penoelo Thibaud
40212017

Lab Instructor: Razieh Abdolahi

Performed on : October 1, 2024
Submitted on : October 22, 2024

I certify that this submission is my original work and meets the Faculty's Expectations of Originality

Objectives –

The purpose of this lab was to implement the Discrete Time Fourier Transform (DTFT) of a pulse input using MATLAB and to introduce Simulink as a modeling and simulation tool within the MATLAB environment. The lab was divided into two parts:

Part I: Focused on implementing the DTFT using MATLAB programming techniques like loops and array manipulations, and plotting the resulting frequency spectrum.

Part II: Aimed at familiarizing students with the basic features of Simulink for simulating discrete-time systems.

Theory -

Part I: This part focused on calculating the DTFT for a discrete signal $x[n]$, which is a crucial concept in analyzing the frequency content of discrete-time signals. The DTFT is mathematically defined as:

$$X(e^{j\omega}) = \sum_{n=-\infty}^{n=+\infty} x[n]e^{-j\omega n}$$

This equation was implemented in MATLAB, allowing the visualization of the frequency response of a signal. The process involved defining the signal $x[n]$, setting the frequency range ω , and using nested loops to compute the DTFT.

Part II: This part introduced Simulink, a MATLAB-based simulation environment. It demonstrated how to simulate systems using blocks such as sources, sinks, and mathematical operators, which is particularly useful for understanding continuous and discrete system responses.

Tasks/Results/Discussion

Questions – Part 1

Question1 :

Obtain the DTFT of a pulse $x[n]$ shown in Figure 2 over the interval $-10 \leq n \leq 10$.

A rectangular pulse

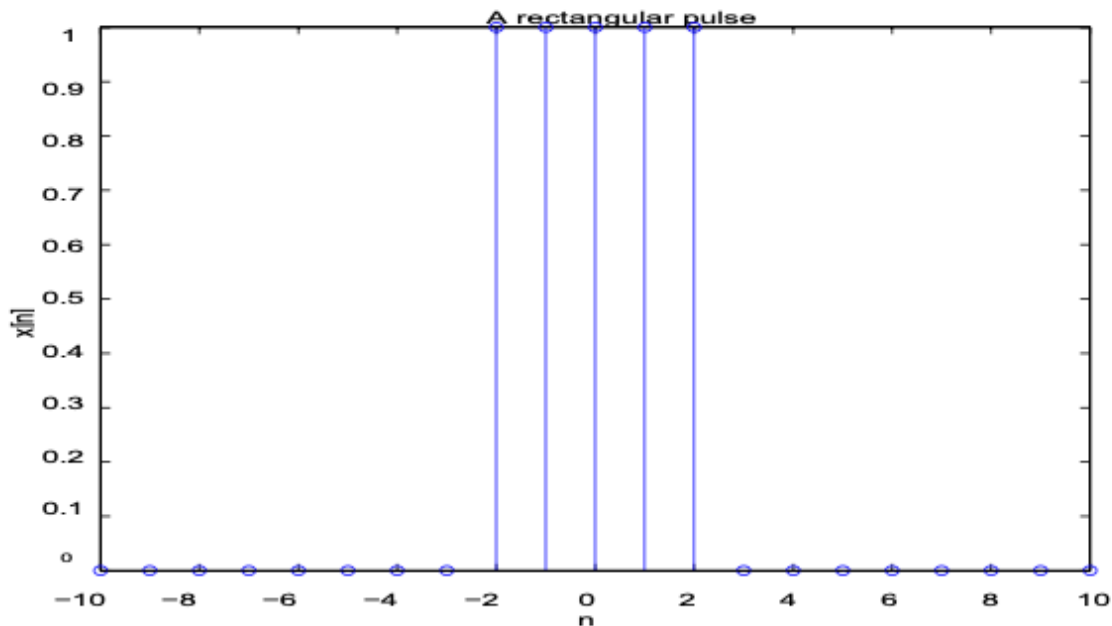


Figure 2: A rectangular pulse.

Compute the transform over the frequency interval $-\pi \leq w \leq \pi$ using a user-defined input value for the step size. Plot the signal $x[n]$ (using stem) together with its transform (using plot). Use the length function to control the number of iterations of the outer for loop given in the pseudocode algorithm.

```
clear
clc
close all

%Define signal n=[-10:10];
w=[-pi:2*pi/100:(pi-2*pi/100)];
length(w);
length(n);
u1=n>=-2;
u2=n>=3;
x=u1-u2;
subplot(1,2,1)
stem(n,x,'color','r','linewidth',2)
```

```

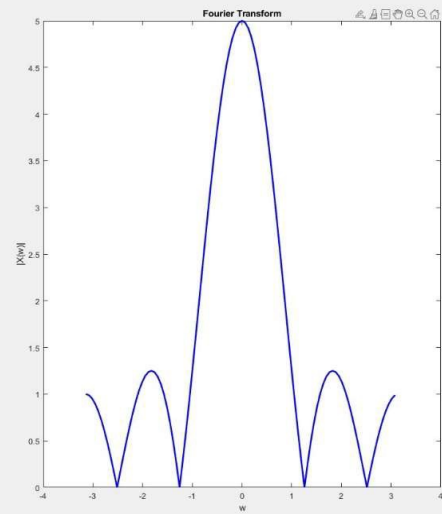
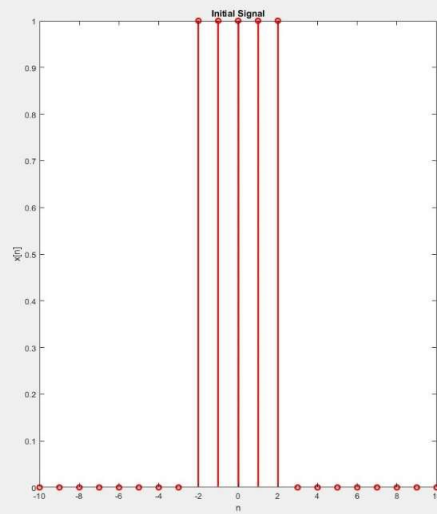
xlabel("n");
ylabel("x[n]");
title("Initial Signal");

%find the fourier transform

for g=1:length(w)
    sum=0;
    for b=1:length(n) sum=sum+(x(b))*(exp(-j*w*b));
    end
end

%find magnitude
m=abs(d);
subplot(1,2,2)
plot(w,m,'linewidth',2,'color','b')
xlabel("w");
ylabel("|X(w)|");
title("Fourier Transform");

```



Question 2 :

MATLAB has a built-in function called `fft` which computes the discrete Fourier transform of an input vector $x(n)$:

```
>> help fft
fft Discrete Fourier transform.
    fft(X) is the discrete Fourier transform (DFT) of vector X.

For length N input vector x, the DFT is a length N vector X,
with elements

$$X(k) = \sum_{n=1}^N x(n) \exp(-j \cdot 2 \cdot \pi \cdot (k-1) \cdot (n-1) / N), \quad 1 \leq k \leq N.$$

```

Re-write your script for Question 1 such that the transform is computed over the interval of $0 \leq w \leq 2\pi$ and compare the results of your transform with that produced by the MATLAB `fft` command. Note that if the input signal $x[n]$ is a vector of length 21 (21 values of n ranging from -10 to +10), then `x_fft = fft(x)` will be a 1 x 21 array. In order to compare the results of `plot(w,x_fft)` with the results of your transform, it will be necessary to choose a step size for w such that the size of the array holding your transform values also has length 21.

```
clear
clc
close all

%Define signal n=[-10:10];
w=[0:2*pi/21:(2*pi-2*pi/21)];
length(w);
length(n);
u1=n>=-2;
u2=n>=3;
x=u1-u2;
subplot(1,3,1)
stem(n,x,'color','r','linewidth',2)
xlabel("n");
ylabel("x[n]");
title("Initial Signal");

%find the fourier transform for g=1:length(w)
sum=0;
for b=1:length(n) sum=sum+(x(b))*(exp(-j*w*b));
end
%store value
d=sum;
end

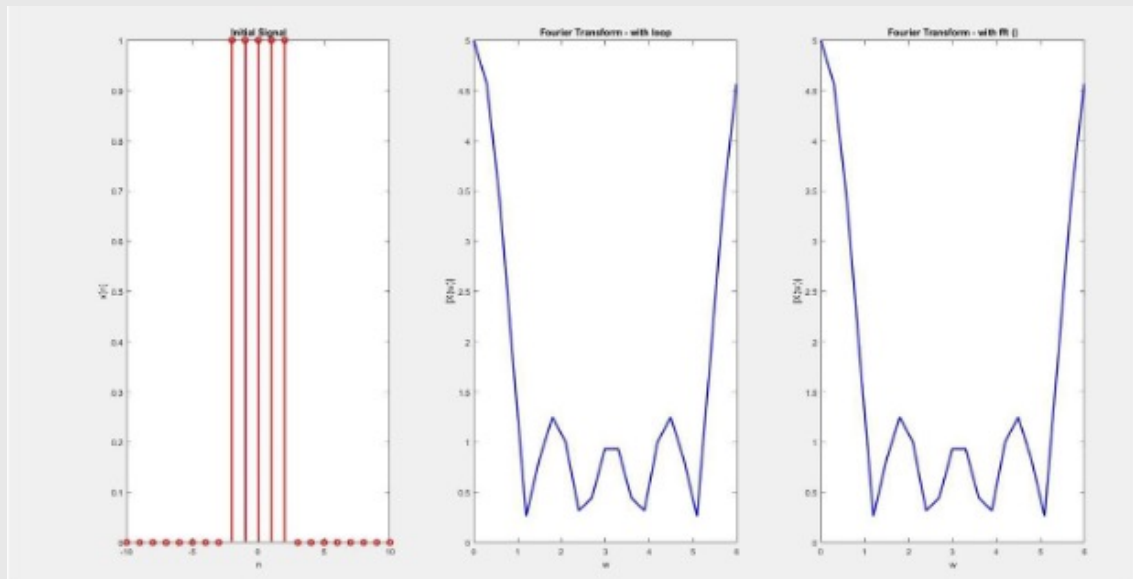
%find magnitude m=abs(d);
```

```

subplot(1,3,2)
plot(w,m,'linewidth',2, 'color','b')
xlabel("w");
ylabel("|X(w)|");
title("Fourier Transform - with loop");

%DTFT with fft
x_fft=abs(fft(x));
subplot(1,3,3)
plot(w,x_fft,'linewidth',2, 'color','b')
xlabel("w");
ylabel("|X(w)|");
title("Fourier Transform - with fft ( )");

```



Question3 :

MATLAB has a function called `ifft` which computes the inverse discrete Fourier transform. Compute the `fft` of the input signal $x[n]$ given in Question 1, and then use it as input to the `ifft` function to obtain the original signal $x[n]$. Plot the original signal $x[n]$ together with the signal obtained from the `ifft` command.

```
clear
clc
closeall

%Define signal n=[-10:10];
w=[0:2*pi/21:(2*pi-2*pi/21)];
length(w);
length(n);
u1=n>=-2;
u2=n>=3;
x=u1-u2;
subplot(1,3,1)
stem(n,x,'color','r','linewidth',2)
xlabel("n");
ylabel("x[n]");
title("Initial Signal");

%DTFT with fft x_fft=fft(x);

subplot(1,3,2)
plot(w,abs(x_fft),'linewidth',2, 'color','b')
xlabel("w");
ylabel("|X(w)|");
title("Fourier Transform - with fft ( )");

%find intial signal with ifft x_ifft=ifft(x_fft);

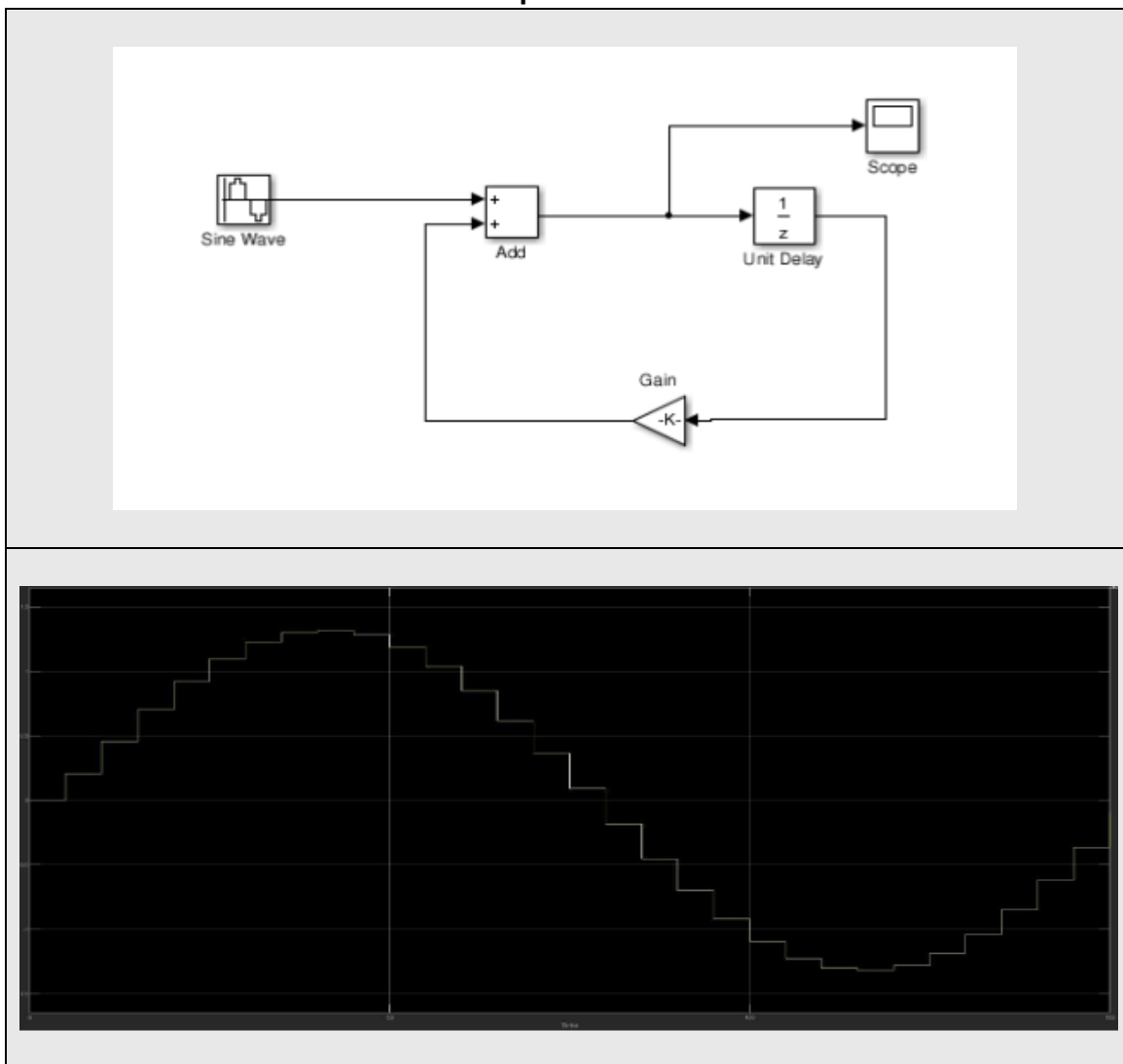
subplot(1,3,3)
stem(n,x_ifft,'linewidth',2, 'color','r') ylim([0 1]);
xlabel("n");
ylabel("x[n]");
title("Initial Signal - with ifft ( )");
```

Questions – Part 2

Question1 :

Use Simulink to simulate the behavior of the difference equation $y[n] = x[n] + y[n - 1]$. Use a sine wave as the input $x[n]$. Decide upon a frequency and sample time of your choice. Use the following Simulink blocks to build the Simulink model of the system as shown in Figure 13:

- an **Integer Delay** block from the **Discrete** library.14
- a **Gain** block from the **Math Operations** library (double click the block to specify the gain parameter as 0.25)
- an **Add** block chosen from the **Math Operations** library
- A **Sine Wave** Source block and a **Scope Sink** Block



Conclusions –

This lab delved into MATLAB-based methods for analyzing discrete-time signals and systems, focusing on various properties such as linearity, symmetry, and time invariance. The experiment was structured into two sections, each targeting different aspects of signal and system analysis.

In Part I, MATLAB's programming features—including loops, conditionals, and array operations—were employed to examine signal characteristics. The results aligned with theoretical predictions, underscoring MATLAB's effectiveness in validating signal properties such as linearity and symmetry.

Part II concentrated on system response analysis through both direct evaluation and convolution techniques. The findings demonstrated that convolution serves as an accurate approach to modeling the behavior of linear time-invariant (LTI) systems, effectively mirroring the outcomes obtained from solving the corresponding difference equations. This section further involved validating the time invariance and linearity of various systems, reinforcing the principles of LTI system analysis.